

★ Review & Rating Management — Complete Guide

PART 1 — Setup

Step 1 — Accept the GitHub Invitation

1. Check your email
 2. Click "**View invitation**"
 3. Click "**Accept invitation**"
-

Step 2 — Make sure you have these installed

- **Git** → <https://git-scm.com/downloads>
 - **IntelliJ IDEA** → <https://www.jetbrains.com/idea/download> (Community is free)
 - **Java 17** → <https://www.oracle.com/java/technologies/downloads/#java17>
-

Step 3 — Clone the project

Open PowerShell and run:

```
git clone https://github.com/LeaderUsername/OnlineToyStore.git  
cd OnlineToyStore
```

Step 4 — Open in IntelliJ IDEA

1. Open IntelliJ IDEA
 2. Click "**Open**"
 3. Select the `OnlineToyStore` folder
 4. Wait for the loading bar at the bottom to finish
 5. If it asks about Maven → click "**Load Maven Project**"
 6. You should see no red errors when it finishes
-

Step 5 — Switch to your branch

In IntelliJ open the **Terminal** tab at the bottom and run:

```
git checkout feature/review-rating
```

Verify:

```
git branch
# Should show:  * feature/review-rating
```

Step 6 — Get the latest code from develop

```
git checkout develop
git pull origin develop
git checkout feature/review-rating
git merge develop
```

PART 2 — Write the Code

File 1 — Update `Review.java`

Go to `src/main/java/com/OnlineToyStore/Sllit/model/Review.java` and **replace everything** with this:

```
package com.OnlineToyStore.Sllit.model;

import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
// Encapsulation — all fields private, accessed via Lombok getters/setters
public class Review {

    private String reviewId;
    private String toyId;
    private String toyName;
    private String userId;
    private String username;
    private int rating;           // 1 to 5
    private String comment;
    private String date;         // format: yyyy-MM-dd
    private String status;       // PENDING, APPROVED, REJECTED

    // Polymorphism — overridden in subclasses
    public String getDisplayFormat() {
        return "[" + status + "] " + username +
            " rated " + rating + "/5 : " + comment;
    }
}
```

```

        // Convert review object → one line in reviews.txt
        // Format:
reviewId|toyId|toyName|userId|username|rating|comment|date|status
        public String toFileString() {
            return reviewId + "|" +
                toyId + "|" +
                (toyName != null ? toyName : "") + "|" +
                (userId != null ? userId : "") + "|" +
                (username != null ? username : "") + "|" +
                rating + "|" +
                (comment != null ? comment.replace("|", "-") : "") + "|" +
                (date != null ? date : "") + "|" +
                (status != null ? status : "PENDING");
        }

        // Read one line from reviews.txt → Review object
        public static Review fromFileString(String line) {
            String[] p = line.split("\\|", -1);
            Review r = new Review();
            r.setReviewId(p[0]);
            r.setToyId(p[1]);
            r.setToyName(p.length > 2 ? p[2] : "");
            r.setUserId(p.length > 3 ? p[3] : "");
            r.setUsername(p.length > 4 ? p[4] : "");
            r.setRating(p.length > 5 ? Integer.parseInt(p[5]) : 0);
            r.setComment(p.length > 6 ? p[6] : "");
            r.setDate(p.length > 7 ? p[7] : "");
            r.setStatus(p.length > 8 ? p[8] : "PENDING");
            return r;
        }
    }
}

```

File 2 — Create `VerifiedReview.java`

Create a new file in the same `model/` folder:

```

package com.OnlineToyStore.Sllit.model;

import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
// Inheritance – VerifiedReview extends Review
public class VerifiedReview extends Review {

    private String orderId; // proof of purchase

    public VerifiedReview(String reviewId, String toyId,
                          String toyName, String userId,
                          String username, int rating,
                          String comment, String date,
                          String orderId) {

```

```

        super(reviewId, toyId, toyName, userId,
            username, rating, comment, date, "APPROVED");
        this.orderId = orderId;
    }

    // Polymorphism – overrides parent getDisplayFormat()
    @Override
    public String getDisplayFormat() {
        return "✅ VERIFIED PURCHASE | " + getUsername() +
            " rated " + getRating() + "/5 : " + getComment();
    }
}

```

File 3 — Create `PublicReview.java`

Create another **new file** in the `model/` folder:

```

package com.OnlineToyStore.Sllit.model;

import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
// Inheritance – PublicReview extends Review
public class PublicReview extends Review {

    private boolean isAnonymous;

    public PublicReview(String reviewId, String toyId,
        String toyName, String userId,
        String username, int rating,
        String comment, String date,
        boolean isAnonymous) {
        super(reviewId, toyId, toyName, userId,
            username, rating, comment, date, "APPROVED");
        this.isAnonymous = isAnonymous;
    }

    // Polymorphism – overrides parent getDisplayFormat()
    // Shows "Anonymous" instead of real username if anonymous
    @Override
    public String getDisplayFormat() {
        String displayName = isAnonymous ? "Anonymous" : getUsername();
        return "🌐 PUBLIC | " + displayName +
            " rated " + getRating() + "/5 : " + getComment();
    }
}

```

File 4 — Create `ReviewService.java`

Create a new file in

src/main/java/com/OnlineToyStore/Sllit/service/ReviewService.java:

```
package com.OnlineToyStore.Sllit.service;

import com.OnlineToyStore.Sllit.model.Review;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.io.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

@Service
public class ReviewService {

    @Value("${data.file.path}")
    private String dataFilePath;

    private String getFilePath() {
        return dataFilePath + "reviews.txt";
    }

    // — READ ALL reviews —————
    public List<Review> getAllReviews() {
        List<Review> reviews = new ArrayList<>();
        File file = new File(getFilePath());
        if (!file.exists()) return reviews;
        try (BufferedReader reader =
            new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (!line.trim().isEmpty()) {
                    reviews.add(Review.fromFileString(line));
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return reviews;
    }

    // — GET reviews by toy ID —————
    public List<Review> getReviewsByToy(String toyId) {
        return getAllReviews().stream()
            .filter(r -> r.getToyId().equals(toyId)
                && "APPROVED".equals(r.getStatus()))
            .collect(Collectors.toList());
    }

    // — GET reviews by user ID —————
    public List<Review> getReviewsByUser(String userId) {
        return getAllReviews().stream()
```

```

        .filter(r -> r.getUserId().equals(userId))
        .collect(Collectors.toList());
    }

    // — GET reviews by status (Admin) —————
    public List<Review> getReviewsByStatus(String status) {
        return getAllReviews().stream()
            .filter(r -> r.getStatus().equalsIgnoreCase(status))
            .collect(Collectors.toList());
    }

    // — GET one review by ID —————
    public Review getReviewById(String reviewId) {
        return getAllReviews().stream()
            .filter(r -> r.getReviewId().equals(reviewId))
            .findFirst()
            .orElse(null);
    }

    // — CALCULATE average rating for a toy —————
    public double getAverageRating(String toyId) {
        List<Review> toyReviews = getReviewsByToy(toyId);
        if (toyReviews.isEmpty()) return 0.0;
        return toyReviews.stream()
            .mapToInt(Review::getRating)
            .average()
            .orElse(0.0);
    }

    // — CREATE — Submit new review —————
    public void submitReview(Review review) {
        review.setReviewId("REV-" +
            UUID.randomUUID().toString()
                .substring(0, 8).toUpperCase());
        review.setDate(LocalDate.now().toString());
        review.setStatus("PENDING");

        try (BufferedWriter writer = new BufferedWriter(
            new FileWriter(getFilePath(), true))) {
            writer.write(review.toString());
            writer.newLine();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    // — UPDATE — Edit review —————
    public void updateReview(Review updated) {
        List<Review> all = getAllReviews();
        List<Review> saved = all.stream()
            .map(r -> r.getReviewId()
                .equals(updated.getReviewId()) ? updated : r)
            .collect(Collectors.toList());
        saveAll(saved);
    }

    // — UPDATE status (Admin approve/reject) —————

```

```

public void updateStatus(String reviewId, String status) {
    List<Review> all = getAllReviews();
    for (Review r : all) {
        if (r.getReviewId().equals(reviewId)) {
            r.setStatus(status);
            break;
        }
    }
    saveAll(all);
}

// — DELETE review —————
public void deleteReview(String reviewId) {
    List<Review> remaining = getAllReviews().stream()
        .filter(r -> !r.getReviewId().equals(reviewId))
        .collect(Collectors.toList());
    saveAll(remaining);
}

// — Count helpers for dashboard —————
public long countByStatus(String status) {
    return getAllReviews().stream()
        .filter(r -> r.getStatus().equalsIgnoreCase(status))
        .count();
}

public long countTotal() {
    return getAllReviews().size();
}

// — Star string helper —————
public String getStarString(int rating) {
    return "★".repeat(Math.max(0, rating)) +
        "☆".repeat(Math.max(0, 5 - rating));
}

// — Private helper —————
private void saveAll(List<Review> reviews) {
    try (BufferedWriter writer = new BufferedWriter(
        new FileWriter(getFilePath(), false))) {
        for (Review r : reviews) {
            writer.write(r.toFileString());
            writer.newLine();
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

File 5 — Create `ReviewController.java`

Create a new file in

`src/main/java/com/OnlineToyStore/Sllit/controller/ReviewController.java`:

```

package com.OnlineToyStore.Sllit.controller;

import com.OnlineToyStore.Sllit.model.Review;
import com.OnlineToyStore.Sllit.service.ReviewService;
import com.OnlineToyStore.Sllit.service.ToyService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/reviews")
public class ReviewController {

    @Autowired
    private ReviewService reviewService;

    @Autowired
    private ToyService toyService;

    // Demo user — replace with session user later
    private static final String DEMO_USER_ID = "USER-001";
    private static final String DEMO_USERNAME = "DemoUser";

    // — All reviews (Admin view) —————
    @GetMapping
    public String listReviews(
        @RequestParam(required = false) String status,
        Model model) {

        if (status != null && !status.isEmpty()) {
            model.addAttribute("reviews",
                reviewService.getReviewsByStatus(status));
            model.addAttribute("currentStatus", status);
        } else {
            model.addAttribute("reviews",
                reviewService.getAllReviews());
            model.addAttribute("currentStatus", "ALL");
        }

        model.addAttribute("totalCount",
            reviewService.countTotal());
        model.addAttribute("pendingCount",
            reviewService.countByStatus("PENDING"));
        model.addAttribute("approvedCount",
            reviewService.countByStatus("APPROVED"));
        model.addAttribute("rejectedCount",
            reviewService.countByStatus("REJECTED"));
        return "review/list";
    }

    // — Submit review form —————
    @GetMapping("/submit")
    public String showSubmitForm(
        @RequestParam(required = false) String toyId,
        Model model) {
        Review review = new Review();

```



```

        if (toyId != null) {
            review.setToyId(toyId);
            var toy = toyService.getToyById(toyId);
            if (toy != null) {
                review.setToyName(toy.getName());
            }
        }
        model.addAttribute("review", review);
        model.addAttribute("toys", toyService.getAllToys());
        return "review/submit";
    }

    // — Handle submit review —————
    @PostMapping("/submit")
    public String submitReview(@ModelAttribute Review review) {
        review.setUserId(DEMO_USER_ID);
        review.setUsername(DEMO_USERNAME);

        // Set toy name from toy list
        if (review.getToyId() != null
            && !review.getToyId().isEmpty()) {
            var toy = toyService.getToyById(review.getToyId());
            if (toy != null) {
                review.setToyName(toy.getName());
            }
        }
        reviewService.submitReview(review);
        return "redirect:/reviews?submitted=true";
    }

    // — View reviews for one toy —————
    @GetMapping("/toy/{toyId}")
    public String viewToyReviews(
        @PathVariable String toyId, Model model) {
        var toy = toyService.getToyById(toyId);
        model.addAttribute("toy", toy);
        model.addAttribute("reviews",
            reviewService.getReviewsByToy(toyId));
        model.addAttribute("avgRating",
            reviewService.getAverageRating(toyId));
        return "review/toy-reviews";
    }

    // — Edit review form —————
    @GetMapping("/edit/{reviewId}")
    public String showEditForm(
        @PathVariable String reviewId, Model model) {
        model.addAttribute("review",
            reviewService.getReviewById(reviewId));
        model.addAttribute("toys",
            toyService.getAllToys());
        return "review/edit";
    }

    // — Handle edit review —————
    @PostMapping("/edit/{reviewId}")
    public String editReview(

```

```

        @PathVariable String reviewId,
        @ModelAttribute Review review) {
    review.setReviewId(reviewId);
    reviewService.updateReview(review);
    return "redirect:/reviews";
}

// — Admin: Approve review —————
@GetMapping("/approve/{reviewId}")
public String approveReview(@PathVariable String reviewId) {
    reviewService.updateStatus(reviewId, "APPROVED");
    return "redirect:/reviews";
}

// — Admin: Reject review —————
@GetMapping("/reject/{reviewId}")
public String rejectReview(@PathVariable String reviewId) {
    reviewService.updateStatus(reviewId, "REJECTED");
    return "redirect:/reviews";
}

// — Delete review —————
@GetMapping("/delete/{reviewId}")
public String deleteReview(@PathVariable String reviewId) {
    reviewService.deleteReview(reviewId);
    return "redirect:/reviews";
}
}

```

File 6 — Create HTML templates

First **create this folder**:

```
src/main/resources/templates/review/
```

Then create these 4 files inside it:

list.html — Admin review moderation page

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Reviews — Online Toy Store</title>
    <link rel="stylesheet" th:href="@{/css/style.css}"/>
    <style>
        .rev-page { max-width:1150px; margin:0 auto; padding:2rem 1.25rem; }
        .stats-row { display:grid; grid-template-columns:repeat(4,1fr);
gap:1rem; margin-bottom:1.75rem; }

```

```

        .stat-box { background:#fff; border-radius:12px; padding:1.1rem
1.25rem; border:1px solid var(--border); box-shadow:0 2px 12px
rgba(27,42,74,0.07); display:flex; align-items:center; gap:0.8rem;
transition:transform 0.2s; }
        .stat-box:hover { transform:translateY(-2px); }
        .stat-box .si { width:44px; height:44px; border-radius:12px;
display:flex; align-items:center; justify-content:center; font-size:1.25rem;
flex-shrink:0; }
        .stat-box h3 { font-size:0.7rem; color:var(--muted); text-
transform:uppercase; letter-spacing:0.5px; font-weight:700; margin-
bottom:2px; }
        .stat-box p { font-family:'Fredoka One',cursive; font-size:1.65rem;
color:var(--secondary); line-height:1; }
        .filter-bar { display:flex; gap:0.5rem; margin-bottom:1.25rem; flex-
wrap:wrap; }
        .filter-btn { padding:6px 16px; border-radius:20px; font-
family:'Nunito',sans-serif; font-weight:700; font-size:0.82rem; text-
decoration:none; border:1.5px solid var(--border); color:var(--text);
background:#fff; transition:all 0.15s; }
        .filter-btn:hover { border-color:var(--primary); color:var(--
primary); }
        .filter-btn.act { background:var(--secondary); color:#fff; border-
color:var(--secondary); }
        .filter-btn.pending.act { background:#FF6D00; border-color:#FF6D00; }
        .filter-btn.approved.act { background:var(--success); border-
color:var(--success); }
        .filter-btn.rejected.act { background:var(--danger); border-
color:var(--danger); }
        .review-card { background:#fff; border:1px solid var(--border);
border-radius:13px; padding:1.25rem; margin-bottom:0.85rem; box-shadow:0 2px
10px rgba(27,42,74,0.07); display:flex; flex-direction:column; gap:0.6rem;
position:relative; transition:box-shadow 0.2s; }
        .review-card:hover { box-shadow:0 4px 20px rgba(27,42,74,0.11); }
        .review-card::before { content:''; position:absolute; left:0; top:0;
bottom:0; width:4px; border-radius:13px 0 0 13px; }
        .review-card.pending::before { background:#FF6D00; }
        .review-card.approved::before { background:var(--success); }
        .review-card.rejected::before { background:var(--danger); }
        .rev-header { display:flex; align-items:center; justify-
content:space-between; flex-wrap:wrap; gap:0.5rem; }
        .rev-user { display:flex; align-items:center; gap:8px; }
        .rev-avatar { width:34px; height:34px; border-radius:50%;
background:linear-gradient(135deg,#667eea,#764ba2); display:flex; align-
items:center; justify-content:center; color:#fff; font-weight:800; font-
size:0.85rem; flex-shrink:0; }
        .rev-username { font-weight:800; font-size:0.92rem; color:var(--
secondary); }
        .rev-date { font-size:0.75rem; color:var(--muted); }
        .rev-stars { color:#FFB300; font-size:1.05rem; letter-spacing:1px; }
        .rev-toy { font-size:0.78rem; color:var(--muted); font-weight:600; }
        .rev-toy span { color:var(--secondary); font-weight:700; }
        .rev-comment { font-size:0.9rem; color:var(--text); line-height:1.55;
}

        .status-badge { padding:3px 10px; border-radius:20px; font-
size:0.72rem; font-weight:800; }
        .badge-pending { background:#FFF3E0; color:#E65100; }
        .badge-approved { background:#E8F5E9; color:#2E7D32; }

```

```

        .badge-rejected { background:#FFE0E0; color:#C62828; }
        .rev-actions { display:flex; gap:6px; flex-wrap:wrap; margin-top:4px;
    }

        .btn-approve { background:#E8F5E9; color:#2E7D32; border:1.5px solid
#A5D6A7; border-radius:8px; padding:5px 12px; font-family:'Nunito', sans-
serif; font-weight:700; font-size:0.8rem; text-decoration:none;
transition:all 0.15s; }
        .btn-approve:hover { background:var(--success); color:#fff; }
        .btn-reject { background:#FFE0E0; color:#C62828; border:1.5px solid
#FFCDD2; border-radius:8px; padding:5px 12px; font-family:'Nunito', sans-
serif; font-weight:700; font-size:0.8rem; text-decoration:none;
transition:all 0.15s; }
        .btn-reject:hover { background:var(--danger); color:#fff; }
        .empty-state { background:#fff; border-radius:13px; padding:4rem
2rem; text-align:center; border:1px solid var(--border); }
        .empty-state .ei { font-size:3.5rem; display:block; margin-
bottom:1rem; }
        @media(max-width:700px){ .stats-row{grid-template-columns:1fr 1fr;} }
    </style>
</head>
<body>

<nav class="navbar">
    <a href="/" class="navbar-brand">🧸 <span>ToyStore</span></a>
    <ul class="navbar-nav">
        <li><a href="/">Dashboard</a></li>
        <li><a href="/users">Users</a></li>
        <li><a href="/toys">Toys</a></li>
        <li><a href="/cart">Cart</a></li>
        <li><a href="/orders">Orders</a></li>
        <li><a href="/reviews" class="active">Reviews</a></li>
        <li><a href="/inventory">Inventory</a></li>
    </ul>
</nav>

<div class="rev-page">

    <!-- Header -->
    <div class="page-header">
        <h1 style="font-family:'Fredoka One',cursive;font-size:1.7rem;
            color:var(--secondary);display:flex;align-
items:center;gap:8px;">
            ★ Review Management
        </h1>
        <a href="/reviews/submit" class="btn btn-primary">👉 Write a
Review</a>
    </div>

    <!-- Stats -->
    <div class="stats-row">
        <div class="stat-box">
            <div class="si" style="background:#F3E5F5;">★</div>
            <div><h3>Total Reviews</h3>
            <p th:text="\${totalCount}">0</p></div>
        </div>
        <div class="stat-box">

```

```

        <div class="si" style="background:#FFF3E0;">⌚</div>
        <div><h3>Pending</h3>
            <p th:text="{pendingCount}"
                style="color:#FF6D00;">0</p></div>
    </div>
    <div class="stat-box">
        <div class="si" style="background:#E8F5E9;">✅</div>
        <div><h3>Approved</h3>
            <p th:text="{approvedCount}"
                style="color:var(--success);">0</p></div>
    </div>
    <div class="stat-box">
        <div class="si" style="background:#FFE0E0;">❌</div>
        <div><h3>Rejected</h3>
            <p th:text="{rejectedCount}"
                style="color:var(--danger);">0</p></div>
    </div>
</div>

<!-- Filter bar -->
<div class="filter-bar">
    <a th:href="@{/reviews}"
        th:class="{currentStatus == 'ALL'} ? 'filter-btn act' : 'filter-
btn'">
        All Reviews
    </a>
    <a th:href="@{/reviews(status='PENDING')}"
        th:class="{currentStatus == 'PENDING'} ?
        'filter-btn pending act' : 'filter-btn pending'">
        ⌚ Pending
    </a>
    <a th:href="@{/reviews(status='APPROVED')}"
        th:class="{currentStatus == 'APPROVED'} ?
        'filter-btn approved act' : 'filter-btn approved'">
        ✅ Approved
    </a>
    <a th:href="@{/reviews(status='REJECTED')}"
        th:class="{currentStatus == 'REJECTED'} ?
        'filter-btn rejected act' : 'filter-btn rejected'">
        ❌ Rejected
    </a>
</div>

<!-- Empty state -->
<div class="empty-state"
    th:if="{reviews == null or reviews.isEmpty()}">
    <span class="ei">★</span>
    <h3 style="font-weight:800;margin-bottom:4px;">No reviews found</h3>
    <p style="color:var(--muted);margin-bottom:1.25rem;">
        No reviews match this filter yet.
    </p>
    <a href="/reviews/submit" class="btn btn-primary">
        📝 Write First Review
    </a>
</div>

```

```

<!-- Review cards -->
<div th:if="${reviews != null and !reviews.isEmpty()}">
  <p style="font-size:0.82rem;color:var(--muted);
    font-weight:600;margin-bottom:1rem;">
    Showing
    <strong style="color:var(--secondary);"
      th:text="${reviews.size()}"></strong>
    review(s)
  </p>

  <div th:each="review : ${reviews}">
    <div th:class="'review-card ' + ${review.status.toLowerCase()}">

      <!-- Top row -->
      <div class="rev-header">
        <div class="rev-user">
          <div class="rev-avatar"
            th:text="${#strings.substring(
              review.username,0,1)
              .toUpperCase()}">

          </div>
          <div>
            <div class="rev-username"
              th:text="${review.username}"></div>
            <div class="rev-date"
              th:text="${review.date}"></div>
          </div>
        </div>
        <div style="display:flex;align-items:center;gap:8px;">
          <span class="rev-stars"
            th:text="${'★'.repeat(review.rating) +
              '☆'.repeat(5 - review.rating)}">

          </span>
          <span th:text="${review.rating} + '/5'"
            style="font-weight:800;font-size:0.88rem;
              color:var(--secondary);">

          </span>
          <span th:text="${review.status}"
            th:class="${review.status == 'APPROVED'} ?
              'status-badge badge-approved' :
              (review.status == 'REJECTED' ?
                'status-badge badge-rejected' :
                'status-badge badge-pending')">

          </span>
        </div>
      </div>
    </div>

    <!-- Toy name -->
    <div class="rev-toy">
      🧸 Toy:
      <span th:text="${review.toyName != null
        and !review.toyName.isEmpty()
        ? review.toyName : review.toyId}">

      </span>
      <span style="color:var(--muted);font-size:0.72rem;"

```

```

            th:text="' (' + ${review.reviewId} + ')'"
        </span>
    </div>

    <!-- Comment -->
    <div class="rev-comment"
        th:text="${review.comment}"
    </div>

    <!-- Actions -->
    <div class="rev-actions">
        <a th:if="${review.status != 'APPROVED'}"

th:href="@{/reviews/approve/{id} (id=${review.reviewId})}"
            class="btn btn-success" >✔ Approve</a>

        <a th:if="${review.status != 'REJECTED'}"

th:href="@{/reviews/reject/{id} (id=${review.reviewId})}"
            class="btn btn-danger" >✖ Reject</a>

        <a th:href="@{/reviews/edit/{id} (id=${review.reviewId})}"
            class="btn btn-outline"
            style="padding:5px 12px;font-size:0.8rem;"
            >✎ Edit
        </a>

        <a
th:href="@{/reviews/delete/{id} (id=${review.reviewId})}"
            class="btn btn-danger"
            style="padding:5px 12px;font-size:0.8rem;"
            onclick="return confirm('Delete this review?')"
            >🗑 Delete
        </a>
    </div>
</div>
</div>
</div>
</div>
</body>
</html>

```

submit.html — Submit review page

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Submit Review — Online Toy Store</title>
    <link rel="stylesheet" th:href="@{/css/style.css}"/>
    <style>
        .star-selector { display:flex; flex-direction:row-reverse;

```

```

        justify-content: flex-end; gap: 4px; margin-top: 4px; }
        .star-selector input { display: none; }
        .star-selector label { font-size: 2rem; color: #ddd; cursor: pointer;
            transition: color 0.15s; }
        .star-selector label:hover,
        .star-selector label:hover ~ label,
        .star-selector input:checked ~ label { color: #FFB300; }
    </style>
</head>
<body>

<nav class="navbar">
    <a href="/" class="navbar-brand">🧸 <span>ToyStore</span></a>
    <ul class="navbar-nav">
        <li><a href="/">Dashboard</a></li>
        <li><a href="/users">Users</a></li>
        <li><a href="/toys">Toys</a></li>
        <li><a href="/cart">Cart</a></li>
        <li><a href="/orders">Orders</a></li>
        <li><a href="/reviews" class="active">Reviews</a></li>
        <li><a href="/inventory">Inventory</a></li>
    </ul>
</nav>

<div class="page">
    <div class="page-header">
        <h1 style="font-family: 'Fredoka One', cursive; font-size: 1.7rem;
            color: var(--secondary); display: flex; align-items: center; gap: 8px;">
            🧸 Write a Review
        </h1>
        <a href="/reviews" class="btn btn-outline">← Back to Reviews</a>
    </div>

    <div class="card" style="max-width: 640px;">
        <form th:action="@{/reviews/submit}" method="post">

            <div class="form-group" style="margin-bottom: 1.1rem;">
                <label>Select Toy</label>
                <select name="toyId" required>
                    <option value="">-- Choose a toy --</option>
                    <option th:each="toy : ${toys}"
                        th:value="${toy.toyId}"
                        th:text="${toy.name}"
                        th:selected="${toy.toyId == review.toyId}">
                </option>
                </select>
            </div>

            <!-- Star rating -->
            <div class="form-group" style="margin-bottom: 1.1rem;">
                <label>Your Rating</label>
                <div class="star-selector">
                    <input type="radio" id="s5" name="rating" value="5"/>
                    <label for="s5" title="5 stars">★</label>
                    <input type="radio" id="s4" name="rating" value="4"/>

```



```

        <label for="s4" title="4 stars">★</label>
        <input type="radio" id="s3" name="rating" value="3"
checked/>

        <label for="s3" title="3 stars">★</label>
        <input type="radio" id="s2" name="rating" value="2"/>
        <label for="s2" title="2 stars">★</label>
        <input type="radio" id="s1" name="rating" value="1"/>
        <label for="s1" title="1 star">★</label>
    </div>
</div>

<div class="form-group" style="margin-bottom:1.5rem;">
    <label>Your Comment</label>
    <textarea name="comment" rows="4"
        placeholder="Share your experience with this
toy..."
        required></textarea>
</div>

<div class="form-actions">
    <button type="submit" class="btn btn-primary">
        ★ Submit Review
    </button>
    <a href="/reviews" class="btn btn-outline">Cancel</a>
</div>

</form>
</div>
</div>

</body>
</html>

```

edit.html — Edit review page

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Edit Review — Online Toy Store</title>
    <link rel="stylesheet" th:href="@{/css/style.css}"/>
    <style>
        .star-selector { display:flex; flex-direction:row-reverse;
                        justify-content:flex-end; gap:4px; margin-top:4px; }
        .star-selector input { display:none; }
        .star-selector label { font-size:2rem; color:#ddd; cursor:pointer;
                        transition:color 0.15s; }
        .star-selector label:hover,
        .star-selector label:hover ~ label,
        .star-selector input:checked ~ label { color:#FFB300; }
    </style>
</head>
<body>

```

```

<nav class="navbar">
  <a href="/" class="navbar-brand">🧸 <span>ToyStore</span></a>
  <ul class="navbar-nav">
    <li><a href="/">Dashboard</a></li>
    <li><a href="/users">Users</a></li>
    <li><a href="/toys">Toys</a></li>
    <li><a href="/cart">Cart</a></li>
    <li><a href="/orders">Orders</a></li>
    <li><a href="/reviews" class="active">Reviews</a></li>
    <li><a href="/inventory">Inventory</a></li>
  </ul>
</nav>

<div class="page">
  <div class="page-header">
    <h1 style="font-family:'Fredoka One',cursive;font-size:1.7rem;
      color:var(--secondary);display:flex;align-
items:center;gap:8px;">
      🖍 Edit Review
    </h1>
    <a href="/reviews" class="btn btn-outline">← Back to Reviews</a>
  </div>

  <p style="color:var(--muted);font-size:0.8rem;
    margin-bottom:1.25rem;font-weight:600;">
    Editing:
    <strong style="color:var(--secondary);"
      th:text="\${review.reviewId}"></strong>
  </p>

  <div class="card" style="max-width:640px;">
    <form th:action="@{/reviews/edit/{id}(id=\${review.reviewId})}"
      method="post">

      <!-- Keep existing fields -->
      <input type="hidden" name="toyId"
        th:value="\${review.toyId}"/>
      <input type="hidden" name="toyName"
        th:value="\${review.toyName}"/>
      <input type="hidden" name="userId"
        th:value="\${review.userId}"/>
      <input type="hidden" name="username"
        th:value="\${review.username}"/>
      <input type="hidden" name="date"
        th:value="\${review.date}"/>
      <input type="hidden" name="status"
        th:value="\${review.status}"/>

      <div class="form-group" style="margin-bottom:0.75rem;">
        <label>Toy</label>
        <input type="text" th:value="\${review.toyName}"
          disabled
          style="background:#F0F4FF;color:var(--muted);
            cursor:not-allowed;">
      </div>
    </form>
  </div>

```

```

<!-- Star rating -->
<div class="form-group" style="margin-bottom:1.1rem;">
  <label>Rating</label>
  <div class="star-selector">
    <input type="radio" id="s5" name="rating" value="5"
      th:checked="${review.rating == 5}"/>
    <label for="s5">★</label>
    <input type="radio" id="s4" name="rating" value="4"
      th:checked="${review.rating == 4}"/>
    <label for="s4">★</label>
    <input type="radio" id="s3" name="rating" value="3"
      th:checked="${review.rating == 3}"/>
    <label for="s3">★</label>
    <input type="radio" id="s2" name="rating" value="2"
      th:checked="${review.rating == 2}"/>
    <label for="s2">★</label>
    <input type="radio" id="s1" name="rating" value="1"
      th:checked="${review.rating == 1}"/>
    <label for="s1">★</label>
  </div>
</div>

<div class="form-group" style="margin-bottom:1.5rem;">
  <label>Comment</label>
  <textarea name="comment" rows="4"
    th:text="${review.comment}"
    required></textarea>
</div>

<div class="form-actions">
  <button type="submit" class="btn btn-primary">
     Save Changes
  </button>
  <a href="/reviews" class="btn btn-outline">Cancel</a>
  <a th:href="@{/reviews/delete/{id} (id=${review.reviewId})}"
    class="btn btn-danger"
    style="margin-left:auto;"
    onclick="return confirm('Delete this review
permanently?')">
     Delete
  </a>
</div>

</form>
</div>
</div>

</body>
</html>

```

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Toy Reviews – Online Toy Store</title>
  <link rel="stylesheet" th:href="@{/css/style.css}"/>
  <style>
    .toy-rev-page { max-width:860px; margin:0 auto; padding:2rem 1.25rem;
  }

    .toy-summary { background:#fff; border-radius:14px; border:1px solid
var(--border); box-shadow:0 2px 14px rgba(27,42,74,0.08); padding:1.5rem;
margin-bottom:1.5rem; display:flex; align-items:center; gap:1.5rem; flex-
wrap:wrap; }
    .avg-score { font-family:'Fredoka One',cursive; font-size:3.5rem;
color:var(--secondary); line-height:1; }
    .avg-stars { color:#FFB300; font-size:1.6rem; letter-spacing:2px; }
    .rev-count { font-size:0.85rem; color:var(--muted); font-weight:600;
  }

    .review-card { background:#fff; border:1px solid var(--border);
border-radius:13px; padding:1.25rem; margin-bottom:0.85rem; box-shadow:0 2px
10px rgba(27,42,74,0.07); }
    .rev-header { display:flex; align-items:center; justify-
content:space-between; margin-bottom:0.6rem; flex-wrap:wrap; gap:0.5rem; }
    .rev-user { display:flex; align-items:center; gap:8px; }
    .rev-avatar { width:34px; height:34px; border-radius:50%;
background:linear-gradient(135deg,#667eea,#764ba2); display:flex; align-
items:center; justify-content:center; color:#fff; font-weight:800; font-
size:0.85rem; }
    .rev-username { font-weight:800; font-size:0.9rem; color:var(--
secondary); }
    .rev-date { font-size:0.73rem; color:var(--muted); }
    .rev-stars { color:#FFB300; font-size:1rem; letter-spacing:1px; }
    .rev-comment { font-size:0.9rem; color:var(--text); line-height:1.55;
  }

    .empty-rev { background:#fff; border-radius:13px; padding:3rem 2rem;
text-align:center; border:1px solid var(--border); }
  </style>
</head>
<body>

<nav class="navbar">
  <a href="/" class="navbar-brand">🧸 <span>ToyStore</span></a>
  <ul class="navbar-nav">
    <li><a href="/">Dashboard</a></li>
    <li><a href="/users">Users</a></li>
    <li><a href="/toys">Toys</a></li>
    <li><a href="/cart">Cart</a></li>
    <li><a href="/orders">Orders</a></li>
    <li><a href="/reviews" class="active">Reviews</a></li>
    <li><a href="/inventory">Inventory</a></li>
  </ul>
</nav>

<div class="toy-rev-page">

```

```

<div class="page-header">
  <h1 style="font-family:'Fredoka One',cursive;font-size:1.6rem;
    color:var(--secondary);">
    ★ Reviews for
    <span th:text="${toy != null} ? ${toy.name} : 'Toy'"></span>
  </h1>
  <div style="display:flex;gap:0.6rem;">
    <a href="/reviews" class="btn btn-outline">← All Reviews</a>
    <a th:href="@{/reviews/submit(toyId=${toy.toyId})}"
      class="btn btn-primary">👉 Write Review</a>
  </div>
</div>

<!-- Rating summary card -->
<div class="toy-summary"
  th:if="${reviews != null and !reviews.isEmpty()}">
  <div>
    <div class="avg-score"
      th:text="${#numbers.formatDecimal(avgRating,1,'POINT',1,'POINT')}">
        0.0
    </div>
  </div>
  <div>
    <div class="avg-stars"
      th:text="${'★'.repeat(avgRating.intValue()) +
        '☆'.repeat(5 - avgRating.intValue())}">
    </div>
    <div class="rev-count"
      th:text="${reviews.size()} + ' approved review(s)'">
    </div>
  </div>
  <div style="margin-left:auto;">
    <a th:href="@{/reviews/submit(toyId=${toy.toyId})}"
      class="btn btn-primary">👉 Write a Review</a>
  </div>
</div>

<!-- Empty state -->
<div class="empty-rev"
  th:if="${reviews == null or reviews.isEmpty()}">
  <span style="font-size:3rem;display:block;margin-
bottom:1rem;">★</span>
  <h3 style="font-weight:800;margin-bottom:4px;">No reviews yet</h3>
  <p style="color:var(--muted);margin-bottom:1.25rem;">
    Be the first to review this toy!
  </p>
  <a th:href="@{/reviews/submit(toyId=${toy.toyId})}"
    class="btn btn-primary">👉 Write First Review</a>
</div>

<!-- Review cards -->
<div th:each="review : ${reviews}" class="review-card">
  <div class="rev-header">
    <div class="rev-user">

```

```

        <div class="rev-avatar"
            th:text="${#strings.substring(
                review.username,0,1).toUpperCase()}">
        </div>
        <div>
            <div class="rev-username"
                th:text="${review.username}"></div>
            <div class="rev-date"
                th:text="${review.date}"></div>
        </div>
    </div>
    <div style="display:flex;align-items:center;gap:6px;">
        <span class="rev-stars"
            th:text="${'★'.repeat(review.rating) +
                '☆'.repeat(5 - review.rating)}">
        </span>
        <span style="font-weight:800;font-size:0.85rem;
            color:var(--secondary);"
            th:text="${review.rating} + '/5'">
        </span>
    </div>
</div>
<div class="rev-comment" th:text="${review.comment}"></div>
</div>
</div>
</body>
</html>

```

PART 3 — Run and Test

`./mvnw spring-boot:run`

Test every page:

URL	What to test
<code>http://localhost:8080/reviews</code>	See all reviews with stats
Click  Pending filter	Only pending reviews show
Click  Approve on a review	Status changes to APPROVED
Click  Reject on a review	Status changes to REJECTED
<code>http://localhost:8080/reviews/submit</code>	Submit a new review with stars
Click  Edit	Update comment and rating
Click  Delete	Review removed
<code>http://localhost:8080/reviews/toy/TOY-XXXXX</code>	See reviews for one toy
Check <code>data/reviews.txt</code>	Data is being saved correctly

PART 4 — Commit and Push

Commit after each file works — do not wait until the end:

```
# After model classes are done
git add .
git commit -m "feat: add Review, VerifiedReview, PublicReview model classes
with file handling"
git push origin feature/review-rating

# After ReviewService is done
git add .
git commit -m "feat: add ReviewService with CRUD, rating calculation, status
management"
git push origin feature/review-rating

# After ReviewController is done
git add .
git commit -m "feat: add ReviewController with submit, approve, reject, edit,
delete routes"
git push origin feature/review-rating

# After all HTML pages are done
git add .
git commit -m "feat: add review list, submit, edit, toy-reviews pages with
star rating UI"
git push origin feature/review-rating

# Final commit after full testing
git add .
git commit -m "feat: complete review and rating management - submit,
moderate, star ratings"
git push origin feature/review-rating
```

PART 5 — Create Pull Request

1. Go to **GitHub**
2. Click "**Compare & pull request**" on feature/review-rating
3. Make sure it says **base: develop** ← NOT main
4. Fill in description:

Component 05 - Review & Rating Management

Built:

- Review.java with toFileString / fromFileString (Encapsulation)
- VerifiedReview.java extending Review (Inheritance)
- PublicReview.java extending Review (Inheritance)
- getDisplayFormat() overridden in both subclasses (Polymorphism)
- ReviewService.java with CRUD, rating average, status management
- ReviewController.java with all routes
- review/list.html with status filter and approve/reject (Admin)

- review/submit.html with interactive star rating selector
- review/edit.html with pre-filled star rating
- review/toy-reviews.html with average rating summary

5. Click **"Create pull request"**

6. **Message the team leader to review and merge**

Important Rules

- ✓ Only work on feature/review-rating
- ✓ Never push to main or develop directly
- ✓ Every day before starting – pull latest develop:

```
git checkout develop
git pull origin develop
git checkout feature/review-rating
git merge develop
```

- ✓ Commit small and often with clear messages
- ✓ Message team leader if you get stuck